

Интерфейс - дизайнерска концепция, която се реализира чрез клас, който съдържа само чисто виртуални функции (=0) без състояние с виртуален деструктор. Методите трябва да са public.

- Дефинира какво може обектът, а не как е реализиран.

Абстрактен клас - интерфейс + базова логика

- не може да се инстанцира
- има поне 1 чисто виртуална функция
- може да има публични данни

- Наследниците имплементират нужните функции и имат ерри и същ публичен деструктор.

Интерфейс - когато има реална нужда от runtime полиморфизъм и типовете се сменят по време на изпълнение

Иножествено наследяване - трябва да е с интерфейси

Object slicing - при копиране по стойност

- Базовият клас не знае за допълнителните ~~класове~~ ^{членове}

Решение: указател или референция

Type casts - изрично преобразуване на тип

static_cast + проверява се по време на компилация
+ без runtime overhead
- не проверява реалния тип на обекта

dynamic_cast + проверява реалния тип на обекта
+ връща nullptr при грешка
+ runtime overhead
- класът трябва да е полиморфен (да има поне 1 virtual функ)

const_cast + не променя типа
+ маха / добавя const
+ работи с указатели и референции

reinterpret_cast - най-опасния cast
- преобразува между несвързани типове указатели

Liskov substitution principle (L in SOLID)

- Обект от наследен клас трябва напълно да замества базовия

Interface Segregation Principle (I in SOLID)

Проблем: - класът "знае" твърде много
- overguide-и с празно тело
- интерфейси с много методи

Решение: ⊕ интерфейси с ясна роля
⊕ имена завършващи на -able

type erasure - техника, при която скриваме конкретния тип на обект и работим с него чрез унифициран интерфейс

Цел: "Не ме интересува типа, важно е какво може да прави."

Цел: Полиморфизъм без наследяване
Runtime гъвкавост

std::function - приема всякакъв callable обект (функция, lambda...)

• Скрива реалния тип, осигурява един общ тип

std::bind - стандартна функция, която създава нов callable обект, като фиксира някои от аргументите на друга функция или преназначаване на тяхната последователност.

Dependency Inversion Principle (D in SOLID)

- Модулите от високо ниво не трябва да зависят от модулите от ниско ниво.
- И двата вида модули зависят от абстракции.